JJBike

1. Objective.

Build a data warehouse so that the administrator of the fictional company JJBike can understand their business.

2. Modeling.

To create the analytical database (OLAP) structure model based on the transactional database (OLTP), using the Star Schema model (the most common model for Business Intelligence), the following transformations were made:

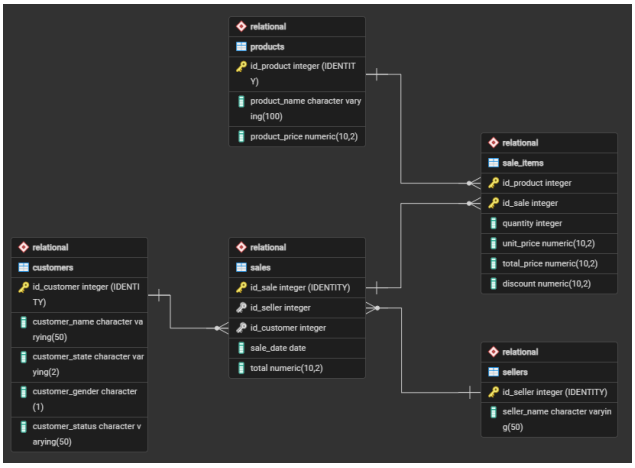
1. Since the goal is to describe the business model, all tables in the OLTP environment, that is, the tables that have one-to-many (1:N) relationships with the table *sales*, were transformed into dimensions:

- **Product ('what?'):** *dim_product*, which holds the attributes of the *products* table from the OLTP environment (excluding *product_price*, as that stays in the fact table), plus the addition of *validity_start_date* and *validity_end_date* for versioning;
- **Seller ('who?'):** *dim_seller*, which holds the attributes of the *sellers* table from the OLTP environment, plus the addition of *validity_start_date* and *validity_end_date* for versioning;
- **Customer ('who?'):** *dim_customer*, which holds the attributes of the *customers* table from the OLTP environment, plus the addition of *validity_start_date* and *validity_end_date* for versioning.

2. **The time dimension ('when?') was created*:*** *dim_time*, which holds the date attribute from the *sales* table in the transactional environment, plus the derived attributes *time_day*, *time_month*, *time_year*, *time_weekday*, and *time_quarter*, created from *time_date*, to enable dimension hierarchies in analyses;

3. In the OLAP environment, the fact table *fact_sales* consolidated the header table (*sales*) and the detail table (*sale_items* (N:M between sales and products)), from the OLTP environment, into a single granularity level: the product item per sale. With its Surrogate Key (SK) as Primary Key (PK), the dimension SKs as Foreign Keys (FK), and the attributes of the junction table *sale_items* as metrics.

- The granularity of the fact table *fact_sales* was defined at the line item per sale level, ensuring the highest level of detail possible for the JJBike administrator's analyses.



3. pgAdmin 4.

In pgAdmin 4, the 'JJBIKE' database was created.

Note: Throughout the SQL code, the Snake Case naming convention will be used.

4. Transactional Environment (OLTP) - pgAdmin 4.

1. The *Relational* schema was created, serving as the data source;

2. The operational tables *sellers*, *products*, *customers*, *sales*, and *sale_items* were created in the *Relational* schema.

- The *GENERATED ALWAYS AS IDENTITY* clause was used to automatically generate the Primary Keys of the tables;
- All columns, except the Primary Keys and *validity_end_date*, were defined with *NOT NULL* constraints, enforcing data integrity at the database level;
- The Referential Integrity Constraints *ON DELETE RESTRICT* and *ON DELETE CASCADE* were used in the *sale_items* entity table, as it is a junction table, ensuring that, respectively:
    - If a record in the *Relational.products* table is attempted to be deleted, and it is being referenced anywhere in the *sale_items* table, the database will raise an error and the deletion will be aborted, preventing sales reports from breaking and orphan data from being created;
    - If a record in the *Relational.sales* table is deleted, the database will automatically delete all related items in the *sale_items* table, as it makes no sense to keep orphan records of a deleted sale.
- Note: The default constraint when defining a FK without specifying a delete or update action is *NO ACTION*, which, if an attempt is made to delete a record in the parent table that has dependents in the child table, blocks the operation and raises an error, preventing the creation of orphan data.

3. Inserts were performed into the tables *sellers*, *products*, *customers*, *sales*, and *sale_items*.

Files used (1. Environments/1. OLTP/):

- **Points 1 and 2:** *1. Scripts_Create_Table_Relational.sql*;
- **Point 3:** *2. Insert_Into_Customers.sql, 3. Insert_Into_Products.sql, 4. Insert_Into_Sellers.sql, 5. Insert_Into_Sales.sql*, and *6. Insert_Into_SaleItems.sql*.

5. Analytical Environment (OLAP) - pgAdmin 4.

5.1. Analytical Database (OLAP)

1. The *Dimensional* schema was created, representing the analytical environment (Data Warehouse (DW)) structured in a Star Schema model;

2. The dimensions *dim_seller*, *dim_customer*, *dim_product*, *dim_time*, and the fact table *fact_sales* were created in the *Dimensional* schema.

- The *GENERATED ALWAYS AS IDENTITY* clause was used to automatically generate the Surrogate Keys (SK) for all dimension and fact tables, serving as the internal primary key of the Data Warehouse (DW);
- All columns, except the Primary Keys, were defined with *NOT NULL* constraints, enforcing data integrity in the analytical layer;
- The concept of Slowly Changing Dimensions (SCD) Type 2 was applied to the sellers, customers, and products dimensions through the columns *validity_start_date* and *validity_end_date*, enabling full versioning and the preservation of data history over time.

3. Inserts were performed into the time dimension. This process was carried out through three main structures:

- The *FROM* block acts as a counter, delivering to the rest of the code a temporary table with a column called *datum* containing all possible dates between 1970 and 2049;
- The *SELECT* block slices this date to extract useful information for analysis;
- The *ORDER BY 1* block ensures that the time dimension table is inserted in perfect chronological order, from the oldest to the most recent date.

4. Using CTEs to organize and simplify complex queries, the following data loads from the transactional database (OLTP) into the analytical database (OLAP) were performed in the described order:

- **Initial Load of customers, products, and sellers:**
  - **CTE *S*:** Selects all data from the source table (e.g.: *Relational.customers*) to serve as a comparison baseline;
  - **CTE *UPD*:** Executes an *UPDATE* on the destination table (*Dimensional*). The logic uses a *WHERE* clause to identify records where the ID matches and *validity_end_date* is null (current record), but some attribute (such as *seller_name* or *customer_status*) has changed. It uses *RETURNING* to list which records were invalidated;
  - **Final *INSERT*:** Inserts new records into the dimension. The condition captures both new records (that do not exist in the dimension) and the new versions of those just invalidated in the *UPD* block.
- **Fact sales load (January, February, March, and remaining months):**
  - **Insertion logic:** The code uses an *INSERT INTO ... SELECT* that consolidates transactional data;
  - **Key mapping (SKs):** Performs *INNER JOIN* between the *sales* and *sale_items* relational tables and the dimensional tables, to acquire the attributes of *sale_items* and the Surrogate Keys of the dimensions, respectively, and assign them to the fact table. The crucial rule is the *validity_end_date IS NULL* filter, which ensures that the sale is associated with the version of the customer/product that was valid at the time of the load;
  - **Time filter:** Uses the function *date_part('MONTH', v.sale_date)* to segment the load month by month.
- **Status Update (SCD Type 2 Simulation):**
  - **Relational change:** A simple *UPDATE* statement is made directly on the source table (*Relational.customers*) to simulate a real-world change;
  - **History processing:** When the customer load block is run again, the database detects that *customer_status* in *S* is different from *customer_status* in *dim_customer*. This

triggers the CTE *UPD* to close the old record and create a new one, preserving the history of what the customer was before becoming 'Gold' or 'Platinum'.

- **Results Verification:**
  - **Audit Queries:** The script ends with *SELECT* combining *fact_sales* with *dim_customer*. This proves that old sales remain linked to the old status (old SK), while new sales point to the updated status (new SK).

Files used (1. Environments/2. OLAP/):

- **Points 1 and 2:** *1. Scripts_Create_Table_Dimensional.sql*;
- **Point 3:** *2. Insert_Into_dim_time.sql*;
- **Point 4:** *3. Script.sql*.

5.2. Cube.

pgAdmin 4 is not a dimensional data server, therefore a denormalized table (a single table with all data) was created that joins the fact table with the dimensions to build the cube. This process was carried out through three main structures:

- The *SELECT* block selects only the columns of interest for analysis (excluding SKs, ID columns, and versioning attributes), extracting the textual attributes from the dimensions and the numerical metrics from the fact table;
- The *INTO* block defines the physical creation of the new table in the database, specifying *Dimensional.sales_cube* as the final destination — a new table in the dimensional schema that will store the unified data;
- The *FROM* block connects the *Dimensional.fact_sales* table to all surrounding dimensional tables through *INNER JOINs*, using the surrogate keys (SKs) to consolidate the star relationship (Star Schema).

Files used (1. Environments/2. OLAP/):

- *4. Cube.sql*.

5.3. KPI (Performance Indicator).

pgAdmin 4 is not a dimensional data server, therefore a denormalized table (a single table with all data) was created to measure total revenue for the fictional company JJBike. This process was carried out through four main structures:

- The *SELECT* block selects only the columns of interest for the macro analysis, extracting the time attribute (month), calculating the aggregated total revenue metric (*SUM*) from the fact table, and dynamically generating the target column through a conditional structure (*CASE*);
- The conversion operator *::numeric*, positioned right after the closing of the conditional block (*END*), acts as a native PostgreSQL shortcut for the *CAST* function. It forces the dynamically generated target column to consolidate as an exact decimal numeric data type, ensuring mathematical standardization and perfect format compatibility with the actual revenue metric when generating achievement calculations in Power BI;
- The *INTO* block defines the physical creation of the new table in the database, specifying *Dimensional.kpi* as the final destination that will store the consolidated results and targets data per month;

- The *FROM* block connects the central *Dimensional.fact_sales* table to the *Dimensional.dim_time* dimension through an *INNER JOIN* using the surrogate keys (SKs), grouping (*GROUP BY*) and ordering (*ORDER BY*) the final results by month.

Files used (1. Environments/2. OLAP/):

- *5. KPI.sql*.

6. Artifacts - Power BI:

1. The tables of the *Dimensional* database, created in pgAdmin 4, were connected to Power BI.

6.1. Reports.

1. The following changes were made to the tables:

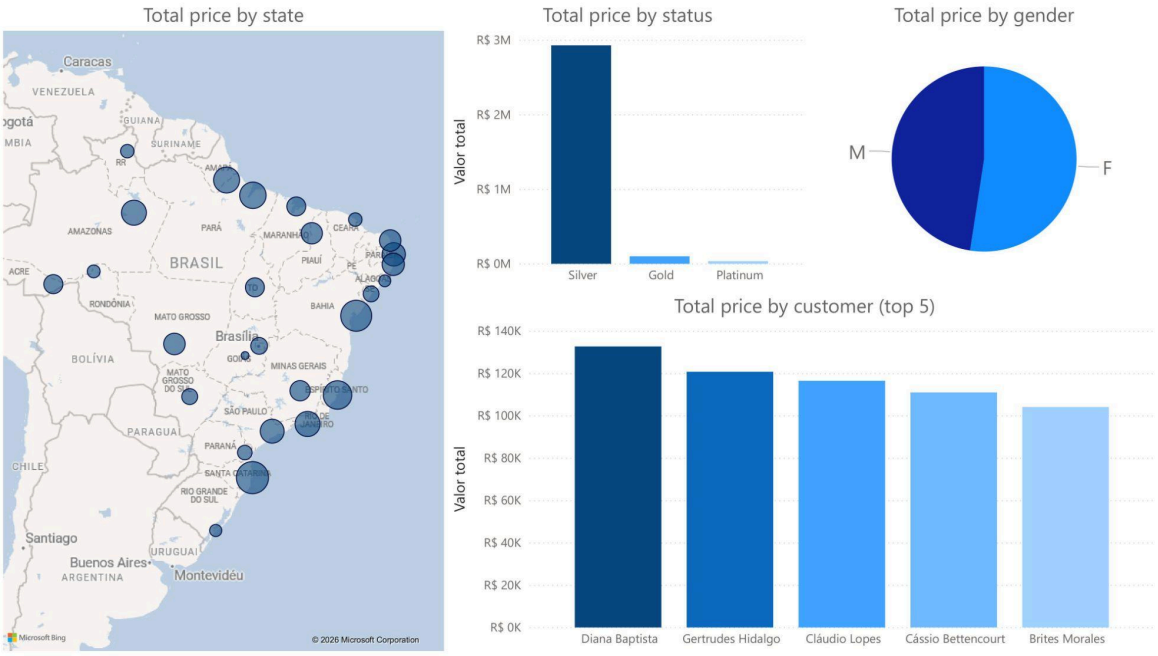
- In *fact_sales*, a column called *discount_percent* (with two decimal places) was created, with the following formula: *discount_percent = TRUNC(('dimensional fact_sales'[discount] / 'dimensional fact_sales'[total_price]) * 100, 2)*;
- In *dim_customer*, a column called *full_location* was created, with the following formula:
  - *location_map =*
    SWITCH(
    'dimensional dim_customer'[customer_state],
    "AC", "Acre", "AL", "Alagoas", "AM", "Amazonas", "AP", "Amapá", "BA", "Bahia", "CE", "Ceará", "DF", "Distrito Federal", "ES", "Espírito Santo", "GO", "Goiás", "MA", "Maranhão", "MG", "Minas Gerais", "MS", "Mato Grosso do Sul", "MT", "Mato Grosso", "PA", "Pará", "PB", "Paraíba", "PE", "Pernambuco", "PI", "Piauí", "PR", "Paraná", "RJ", "Rio de Janeiro", "RN", "Rio Grande do Norte", "RO", "Rondônia", "RR", "Roraima", "RS", "Rio Grande do Sul", "SC", "Santa Catarina", "SE", "Sergipe", "SP", "São Paulo", "TO", "Tocantins", ‘‘Unknown’’
) & ", Brazil".
- In *dim_kpi*, three metrics were created so that, in the KPI report, the card behaves dynamically according to what is selected in the button slicer:
- In *dim_kpi*, three metrics were created so that, in the KPI report, the card's behavior is dynamic according to what you selected in the button slice:
  - *measure_target =*
    IF(
    ISFILTERED('dimensional kpi'[month]) || ISFILTERED('dimensional dim_time'[time_year]),
    SUM('dimensional kpi'[target]),
    0
).
  - *measure_total_revenue =*
    IF(
    ISFILTERED('dimensional kpi'[month]) || ISFILTERED('dimensional dim_time'[time_year]),
    SUM('dimensional kpi'[total_revenue]),
    0

).

2. The following reports were created:

(Customers)

The customers report shows: total value sum by state, total value sum by status, total value sum by gender, and total value sum by customer (top 5).



(Sellers)

The sellers report shows: total value sum by seller (top 5 best sellers, top 5 worst sellers, and top 5 best sellers compared month by month).

(Products)

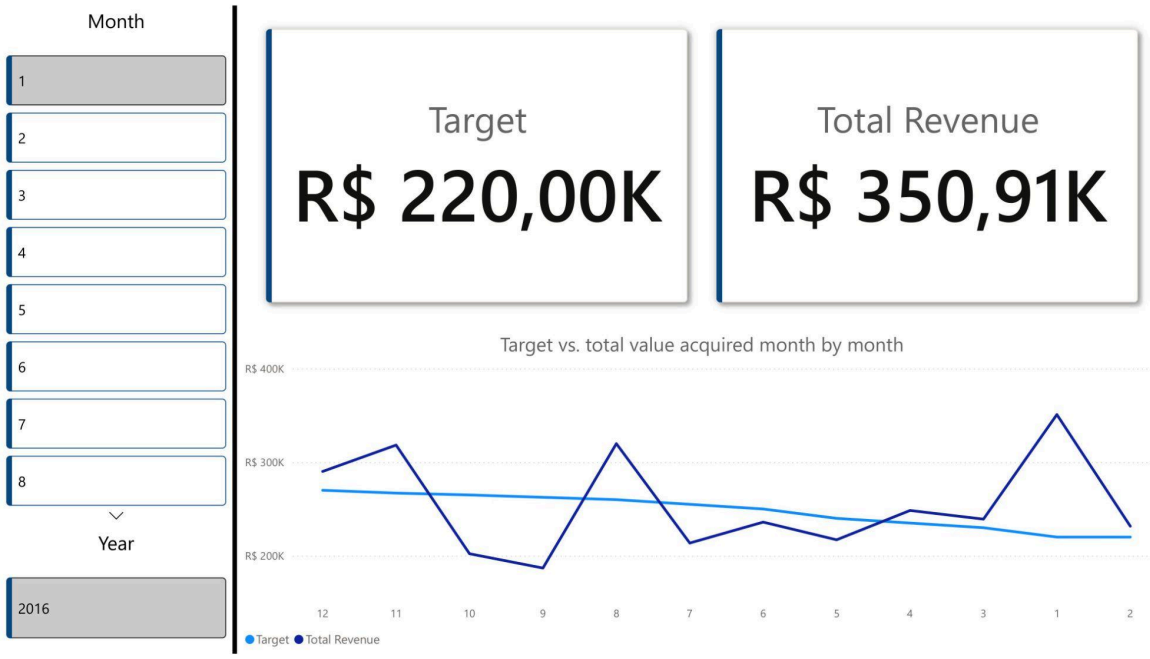
The products report shows: total value sum by product (top 5 best-selling products and top 5 least-selling products) and discount sum by product (top 5 products with the most accumulated discount and top 5 with the least accumulated discount).

(Sales)

The sales report shows: total value sum of products sold (compared month by month), quantity sum of products sold (compared month by month), discount sum of products sold (compared month by month), and product, seller, and customer cards for filtering.

(KPI)

The KPI report shows: target, total acquired value, sum of target and total acquired value (compared month by month), and month and year cards for filtering.



6.2. Cube.

1. The following hierarchies were created from the dimensions:

- ***Geographic Hierarchy:*** *customer_state* → *customer_name* (*dim_customer*);
- ***Date Hierarchy:*** *time_year* → *time_quarter* → *time_month* → *time_day* (*dim_time*);
- Notes:
  - *customer_gender* and *customer_status* (from *dim_customer*) should remain free in the side panel as Loose Dimension Attributes, as placing them inside *Geographic Hierarchy* would imply they are part of a geographic subdivision;
  - *dim_seller* and *dim_product* have only one attribute beyond their keys, therefore they did not generate hierarchies, and their attributes became Loose Dimension Attributes.

2. Visual representation of the created cube:

Customer State	Customer Name	Year	Quantity	Discount	Total Price
⊟ AC	⊟ Alarico Quinterno	⊞ 2016	1	R$ 60,45	R$ 155,00
		Total	1	R$ 60,45	R$ 155,00
	⊞ Antônio Sobral		3	R$ 3.334,61	R$ 16.787,60
	⊞ Basilio Soares		18	R$ 2.516,00	R$ 42.237,47
	⊞ Carla Briones		1	R$ 587,75	R$ 2.260,58
	⊞ Ester Castanho		5	R$ 2.327,33	R$ 15.416,98
	⊞ Evaristo Bahía		3	R$ 692,17	R$ 4.836,70
	⊞ Irene Villanueva		3	R$ 1.654,34	R$ 7.183,75
	Total		34	R$ 11.172,65	R$ 88.878,08
⊞ AL			6	R$ 4.199,33	R$ 26.817,61
⊞ AM			57	R$ 15.881,10	R$ 168.571,45
⊞ AP			53	R$ 16.202,60	R$ 183.945,80
⊞ BA			65	R$ 18.104,59	R$ 274.690,93
⊞ CE			11	R$ 9.146,31	R$ 37.562,17
⊞ DF			19	R$ 14.839,02	R$ 65.272,18
⊞ ES			89	R$ 12.590,55	R$ 226.925,84
⊞ GO			3	R$ 858,20	R$ 7.614,38
⊞ MA			38	R$ 5.149,81	R$ 87.431,06
⊞ MG			31	R$ 18.853,70	R$ 102.980,97
⊞ MS			19	R$ 11.094,30	R$ 57.340,96
⊞ MT			44	R$ 11.279,89	R$ 119.272,51
⊞ PA			82	R$ 24.476,00	R$ 190.925,91
⊞ PB			51	R$ 8.791,16	R$ 143.634,26
⊞ PE			42	R$ 19.477,64	R$ 129.473,93
⊞ PI			44	R$ 20.851,47	R$ 116.457,99
⊞ PR			22	R$ 5.322,31	R$ 45.718,33
⊞ RJ			57	R$ 19.318,44	R$ 169.913,05
⊞ RN			45	R$ 16.227,19	R$ 118.559,35

Generated files available in '2. Artifacts/'.